



Python Coursework

For each logical operation described in the tasks:

- Implement a separate function for each task
- Use descriptive function names that reflect the task being performed.
- Each function must return the result instead of printing it directly.
- The main block (if `__name__ == "__main__":`) should call the functions and print results clearly.
- Use only standard Python libraries and the required libraries (NumPy, Pandas, Matplotlib) where appropriate.
- Do not hardcode predefined results or outputs in the program.
- The program should execute correctly without modifying the source code.
- Use `input()` only where user input is required. Tasks involving random number generation or dataset loading should not use `input()`.
- Ensure that all arrays and variables are created programmatically according to the instructions.
- The code should be well-structured, readable, and logically organized.

Task 1: Operations in Numpy

Problem

1: Numpy array creation and manipulation

- 1.1 Create a 1D NumPy array "a" of length 10 containing random integers between 0 and 99 (inclusive).
- 1.2 Create a 2D Numpy array "b" of shape (3, 4) containing random integers between -10 and 10.
- 1.3 Flatten the array "b" into a 1D NumPy array named "b_flat"



- 1.4 Create a copy of “a”, name it “a_copy”, and set the first element of “a_copy” to -1.
- 1.5 Create a 1D Numpy array “c” containing every second element of “a” starting from index 0.

2: Numpy array indexing and slicing

- 2.1 Print the third element of “a”
- 2.2 Print the last element of “b” (bottom-right element).
- 2.3 Print the first two rows and last two columns of “b”.
- 2.4 Assign the second row of “b” to a variable called “b_row”.
- 2.5 Assign the first column of “b” to a variable called “b_col”.

3: Numpy array operations

- 3.1 Create a 1D Numpy array “d” containing the integers from 1 to 10.
- 3.2 Add “a” and “d” element-wise to create a new Numpy array “e”.
- 3.3 Multiply “b” by 2 to create a new Numpy array “b_double”.
- 3.4. Calculate the matrix multiplication of “b” and the transpose of “b_double” and store the result in f.
- 3.5 Calculate the mean of all elements in a, b, and b_double separately and store them in a NumPy array “g” of length 3.

4: Numpy array aggregation

- 4.1 Find the sum of every element in “a” and assign it to a variable “a_sum”.
- 4.2 Find the minimum element in “b” and assign it to a variable “b_min”.
- 4.3 Find the maximum element in “b_double” and assign it to a variable “b_double_max”.

Task 2: Pandas

Problem

Dataset: <https://www.kaggle.com/datasets/rkiattisak/sports-car-prices-dataset>

1. Load the dataset into a Pandas DataFrame and display the first 5 rows to get an idea of the data.
2. Use Pandas to clean the dataset by removing any missing or duplicate values, and converting any non-numeric data to numeric data where appropriate.
3. Use Pandas to explore the dataset by computing summary statistics for each column, such as mean, median, mode, standard deviation, and range (**Range should be calculated as (max – min)**)
4. Use Pandas to group the dataset by car make and compute the average price for each make.
5. Use Pandas to group the dataset by year and compute the average horsepower for each year.
6. Use Pandas to create a scatter plot of price versus horsepower, and add a linear regression line to the plot (Use NumPy polyfit() to add a regression line.)
7. Use Pandas to create a histogram of the 0-60 MPH times in the dataset, with bins of size 0.5 seconds.
8. Use Pandas to filter the dataset to only include cars with a price greater than \$500,000, and then sort the resulting dataset by horsepower in descending order.
9. Use Pandas to export the cleaned and transformed dataset to a new CSV file.